

04_implementation_plan

HelixLLM Full-Extension — Implementation Plan (Phase P)

Document	Master implementation plan (phases → tasks → subtasks)
Revision	1 · Created 2026-07-06
Status	PLAN READY — pending operator go for Phase 0
Basis	13 research reports + 02_cross_cutting_foundations_ADR.md + 99_risk_bottleneck_analysis.md
Constitution HEAD	0882b9e (through §11.4.182)
Track/branch	(T1/main) → implementation branch minted per §11.4.181 (one canonical name)

Operating rules for every task (bind once, apply everywhere)

1. **Subagent-driven** (§11.4.70); each change **independently reviewed** to a clean GO (§11.4.125/§11.4.142/§11.4.134); **multi-angle impact research** per change (§11.4.145).
2. **Reproduce-first tests** (§11.4.146 STEP1 → 2 → 3): RED-on-broken-artifact polarity test (§11.4.115) → GREEN confirm → extend-to-all-cases; register a standing regression guard (§11.4.135).
3. **Runtime-signature = definition of done** (§11.4.108): a task is done only when its declared machine-checkable signature verifies on a **clean deploy** — *build-success* $\neq$ *works*. No metadata/config/absence-of-error PASS (§11.4/§11.4.1).
4. **Rock-solid captured proof** (§11.4.5/§11.4.69/§11.4.107/§11.4.123); unclear-how-to-test $\Rightarrow$ deep research first (§11.4.150), never a bluff.
5. **Rootless podman via the containers submodule** (§11.4.76/§11.4.161); **no CI** (§11.4.156); **no force-push, merge-onto-latest** (§11.4.113); **no silent removal** (§11.4.122) / investigate-before-remove (§11.4.124).
6. **No hardcoded hosts/paths/content** (CONST-045/046/051); large weights gitignored + §11.4.77 re-obtain script.

7. Every agent/work-stream label carries (T<N>/<branch>) (§11.4.182).

Critical-path sequencing (from stream 99 — hard serial prefix, then parallel)

P0 host GPU foundation → P1 serving core + VRAM broker → P2 gateway +
(nothing downstream is trusted until P0 prints nvidia-smi in a rootless
container AND P1 proves REAL inference on the card)

┌ P3 extended capabilities (vision/gen/xlat
├ P4 provider adapters + protocols (A2A / M
├ P5 codegraph + opendesign core wiring
└ P6 claude_toolkit + LLMsVerifier update



P7 setup + PATH install → P8 HelixQA + full te

PHASE 0 — Host GPU foundation (UNBLOCKING PREFIX — owns DZ-01/DZ-02/G-HOST-1/2/3)

Goal: a rootless-podman container can run GPU work on the RTX 5090, and
llama.cpp/vLLM are built for sm_120 with a REAL inference proof. **Everything
else is blocked on this.**

- **P0-T1 — Install NVIDIA Container Toolkit on ALT-Linux (V-03 first).**
 - Subtasks: verify ALT-Linux packaging (nvidia-container-toolkit may not be in apt/dnf — check NVIDIA rpm repo / from-source nvidia-ctk; **this is UNCONFIRMED — verify before assuming**); nvidia-ctk config --set nvidia-container-cli.no-cgroups --in-place; nvidia-ctk cdi generate --output=\$HOME/.config/cdi/nvidia.yaml; nvidia-ctk cdi list.
 - **Acceptance signature (§11.4.108/§11.4.69):** podman run --rm --device nvidia.com/gpu=all --security-opt=label=disable nvidia/cuda:12.8.1-base-ubuntu22.04 nvidia-smi prints “RTX 5090” — captured to docs/qa/<run>/p0_gpu_passthrough.txt. Owns DZ-01.
 - Deps: none. Risk: ALT-Linux packaging (V-03); rootless CDI may fail where privileged works (podman #17539) — document fallback.
- **P0-T2 — Pin the CUDA-12.8 base image + build-container (G-HOST-2).**
 - Tracked docker/Dockerfile.cuda-base (or containers-submodule base spec) FROM nvidia/cuda:12.8.x-devel; torch 2.9 cu128; TORCH_CUDA_ARCH_LIST=12.0; VLLM_FLASH_ATTN_VERSION=2. nvcc lives in the container only (never host).

- **Signature:** `podman run <base> nvcc --version` shows 12.8; a `torch.cuda.is_available() + torch.cuda.get_device_name()` prints the 5090.
- **P0-T3 — Build llama.cpp for sm_120 (G-HOST-3) + vLLM source build (DZ-02).**
 - llama.cpp: `cmake -DGGML_CUDA=ON -DCMAKE_CUDA_ARCHITECTURES=120` in `dependencies/LLama_CPP` (inside the CUDA container); produce `llama-server`.
 - vLLM: source build against the pinned `torch-nightly/CUDA-12.8` (prebuilt wheels fail on `sm_120`).
 - **Signature:** a real `/v1/completions` request to each server returns non-empty text AND `nvidia-smi` shows a VRAM delta — captured. *Build-success is NOT acceptance.* Owns DZ-02/CX-01.
- **P0-T4 — Driver decision (V — 570.169 vs ≥575).** Investigate whether `stream-01's` ≥575 recommendation is required for vLLM `sm_120`; if a driver change is needed it is host-safety-gated (§11.4.133/§12) and operator-confirmed. Capture the decision + evidence.

PHASE 1 — Serving core + VRAM residency broker (owns DZ-03/DZ-04/DZ-05)

- **P1-T1 — Confirm & specify the HelixLLM API contract (DZ-05, §11.4.73 main-spec).**
 - Read `submodules/helix_llm/internal/brain/*` + handlers; author `submodules/helix_llm/docs/API_CONTRACT.md` (the canonical spec) enumerating every endpoint + request/response JSON (chat/completions/models/embeddings/messages + health). **Blocks HelixQA banks + gateway routing — do FIRST in P1.** No guessing (§11.4.6).
- **P1-T2 — Pin HelixLLM as a single root submodule (gap #3 / drift).**
 - Resolve the two-checkout drift (`submodules/helix_llm/` + `dependencies/HelixDevelopment/helix_llm/`) — investigate-before-remove (§11.4.124); add ONE `.gitmodules` gitlink; ask operator before removing either checkout (§11.4.122).
- **P1-T3 — Primary fleet model live: Qwen3-Coder-30B-A3B (MoE).**
 - GGUF (llama-server) or AWQ (vLLM) per P0-T3; launch flags from ADR (`--parallel 12 --cont-batching -fa --cache-type-k/v q8_0` OR `vLLM --max-num-seqs 12 --enable-prefix-caching --enable-auto-tool-choice --tool-call-parser qwen3_coder`).
 - **Signature:** 12 concurrent `/v1/chat/completions` streams each ≥30 tok/s, VRAM within budget — measured on-card (re-measure, don't trust blog numbers CX-04), captured.

- **P1-T4 — DESIGN SPIKE: VRAM residency broker (DZ-04 — new component, design before build).**
 - Design doc submodules/helix_llm/docs/VRAM_BROKER.md: tiered residency (resident / warm-swappable via vLLM Sleep Mode or Ollama keep_alive:0 / burst single-owner §11.4.119 / CPU-only); a VRAM budget-broker admission API; eviction policy; per-engine caps. THEN implement in internal/brain/ (residency scheduler).
 - **Signature:** load VLM on-demand while the coder fleet stays live; broker refuses admission when budget exceeded (no OOM under a scripted burst); captured nvidia-smi timeline.

PHASE 2 — Gateway + LLMsVerifier extension (owns CONST-036–040 anti-bluff)

- **P2-T1 — Extend HelixAgent’s OpenAI/Anthropic REST server as the unified /v1 gateway** (ADR Decision 4; reuse not reimplement §11.4.74). Route per capability to HelixLLM/containers; advertise /v1/models + capabilities.
- **P2-T2 — LLMsVerifier: add Message.ToolCalls field (highest-leverage, stream 10).** Unblocks tool-calling/MCP verification. RED test proving current (false,0) → GREEN.
- **P2-T3 — Capability probes fail-closed (CONST-040 anti-bluff).** Demote registry.go static MCP/LSP/ACP flags to seed-only; add MCP/LSP/ACP/RAG/Skills/Plugins + CapabilityEvidence to ModelCapability + DB columns; each flag set by ONE real probe with a captured wire artefact; golden-good/bad self-validated analyzers. Owns the static-registry anti-bluff hazard.
- **P2-T4 — Local HelixLLM verification + freshness.** Register HelixLLM endpoint via a config-driven ProviderDescriptor; bind 24h stale re-queue (CONST-037) + ≤60s poll (CONST-038) on the existing event_stream seam.

PHASE 3 — Extended capabilities (parallel after P2; each = container + gateway route + verifier probe + HelixQA bank)

Per-capability task template (T-a...T-h): boot container (P0 base) → gateway route → LLMsVerifier probe → HelixQA bank (P8) → captured proof.

- **P3-T1 Vision (VLM):** Qwen3-VL-8B/30B via llama.cpp+mmproj (- -jinja for tool_calls) or vLLM; /v1/chat/completions image parts. Signature: image → correct answer vs golden fixture + self-validated analyzer (§11.4.107(10)).

- **P3-T2 Image gen:** FLUX.1-dev/schnell via ComfyUI; /v1/images/ generations bridge. Signature: real image, not stub (dimensions + non-trivial content check).
- **P3-T3 Video gen (WAN 2.2 + LTX):** ComfyUI; async job endpoint. Signature: ffmpeg frame-advance + codec/resolution/fps (§11.4.107) on the produced mp4.
- **P3-T4 Vectorize:** vtracer (+ StarVector-8B) FastAPI /vectorize. Signature: raster → valid SVG that re-rasterizes to match within tolerance.
- **P3-T5 Translation:** NLLB-200-3.3B (CTranslate2 int8, warm) + TOWER+ 9B (vLLM, on-demand); LibreTranslate-compatible /translate + doc/glossary. Signature: COMET + back-translation metamorphic (guard COMET-gaming CX-05).
- **P3-T6 Embeddings + reranker:** Qwen3-Embedding-4B (+BGE-M3 sparse) + bge-reranker-v2-m3 via TEI; /v1/embeddings + rerank. Signature: real vectors + retrieval Recall@K on a fixture.
- **P3-T7 RAG + HelixMemory:** Qdrant + code-RAG (tree-sitter chunk → hybrid → codegraph graph-expand → rerank) + Zep/Graphiti + mem0 exposed via MCP. Signature: injected-fact recall + faithfulness.
- **P3-T8 STT + OCR:** faster-whisper (+Parakeet streaming) /v1/audio/transcriptions; Tesseract 5.5 image_to_data unified /v1/ocr (per-word conf+bbox — feeds §11.4.117/§11.4.137). Signature: WER/CER on fixtures; OCR conf-floor + ROI.

PHASE 4 — Provider adapters + protocols (parallel after P2)

- **P4-T1 New provider adapters** (HelixAgent internal/llm/providers/ + LLMsVerifier registration), all CONFIRMED public APIs: **Poe, Perplexity/ Sonar, Sakana Fugu, Xiaomi MiMo, Tencent Hunyuan** (Yuanbao=app → target Hunyuan), **xAI Grok, Moonshot Kimi, Zhipu GLM, Fireworks, DeepInfra, Novita, AI21, Reka**; verify Hyperbolic/Baseten base URLs at build. Each: adapter + LLMsVerifier probe + live round-trip proof (§11.4.69).
- **P4-T2 GPT-Sol** — add OpenAI GPT-5.6 “Sol” model IDs to the existing OpenAI adapter when GA (preview-pending flag). **No new adapter** (operator C1).
- **P4-T3 Qwythos 9B / GOT-OCR2.0** — self-host via the local/Ollama/HF path (Qwythos = chat; GOT-OCR2.0 = an OCR engine option in P3-T8). No hosted adapter.
- **P4-T4 Google A2A protocol** (operator C3) — implement A2A agent ↔ agent interop in HelixAgent (Go; verify SDK/spec latest §11.4.99). Editor/client auto-recognition served separately by the /v1/models + MCP server/discover capability surface.

- **P4-T5 MCP** — official Go SDK github.com/modelcontextprotocol/go-sdk; gateway as MCP server (Helix tools) + host (codegraph/opendesign); stateless-first (2026-07-28 RC); verify Streamable-HTTP server transport availability (UNCONFIRMED).
- **P4-T6 OKF-via-MCP** (operator C2) — implement Google Open Knowledge Format as the on-disk RAG/Skills knowledge format, served through an MCP resources server.
- **Deferred:** Subquadratic — BLOCKED-until-GA (operator C4).

PHASE 5 — codegraph + opendesign core wiring (owns CONST-045 defect)

- **P5-T1** Fix `.mcp.json` rot: bare `codegraph serve --mcp` (no macOS path), exclude vendored third-party (§11.4.79b — drop 6 GB bloat), reconcile 1.1.1 ↔ 1.2.0, wire weekly `codegraph_update_and_resync.sh` (§11.4.80). PreToolUse guard grepping for absolute host paths (§11.4.109-class).
- **P5-T2** Enable opendesign MCP (`npm open-design-mcp`, :7456 daemon, BYOK from `.env` §11.4.10); resolve `od` ↔ `coreutils` collision; health-gate + honest SKIP.
- **P5-T3** `codegraph` = CodeIntel provider feeding `internal/repomap/context/cognee/memory` + HelixAgent `internal/rag`; impact/affected scopes `edits+tests` (§11.4.145/§11.4.108).
- **P5-T4** `opendesign` = design-system provider: `design-systems/helixcode/` (DESIGN.md + `tokens.css` light+dark) governs TUI (`internal/theme`), Fyne desktop, web (§11.4.162; visual-regression tests).

PHASE 6 — claude_toolkit + LLMsVerifier update (SIBLING repo)

- **P6-T1** `detect_helixagent()` gated on command `-v helixagent` in `cmd_sync`; emit resolved-shaped record via `jq` (`transport=router`; `base_url=http://localhost:8100/v1`; `strong=helix-debate`, `fast=helix-llm`); model enum via live `GET /v1/models`.
- **P6-T2** `scripts/tests/verify_helixagent_live.sh` mirroring `verify_providers_live.sh` — `PATH+record+no-secret` asserts, live models, **real /v1/chat/completions round-trip** (§11.4.69), negative case; hermetic `test_providers.sh` with a fake `helixagent`.
- **P6-T3** Update vendored LLMsVerifier `17b4bfb6` → `0e7d6949` (ff, §11.4.113), **rebuild bin/model-verification + .local-cache/code-verification** (avoid SOURCE → ARTIFACT bluff §11.4.108), re-run verifier tests. Extended docs/guides/diagrams for all new aliases.

PHASE 7 — Setup + PATH install

- **P7-T1** `setup.sh` (+ per-subsystem installers) that build + install to PATH: **HelixCode**, **HelixAgent**, **HelixLLM**, **LLMsVerifier** (+ others); wire P0 GPU prereqs, `fetch_weights.sh/fetch_models.sh/fetch_tessdata.sh` re-obtain (§11.4.77), `codegraph/opendesign`, container boot. Signature: `fresh-shell which helixcode helixagent helixllm llmsverifier all resolve + --version`.

PHASE 8 — HelixQA + full test matrix + Challenges

- **P8-T1** New HelixQA test banks per capability (LLM/vision/image/video/translation/embed/RAG/STT/OCR/provider-coverage) using the existing `ContentAssertingResolver` + `Conduit` + `pkg/recordingqa`; each PASS gated on a `*_verdict.json` content assertion + paired §1.1 mutation + self-validated golden-good/bad analyzer.
- **P8-T2** Full test-type matrix (CONST-050): unit/integration/e2e/security/**stress+chaos** (§11.4.85)/perf + **Challenges** (challenges).
- **P8-T3** Autonomous HelixQA session (`helixqa autonomous --banks ...`), live-watched via `conduit-monitor`; acceptance = self-driving -count=3, curated evidence under `docs/qa/<run-id>/` (§11.4.83), window-scoped project-prefixed recordings (§11.4.153–155/§11.4.158–160), media-validation (§11.4.163).

PHASE 9 — Live run + resumption

- **P9-T1** Bring up the full stack (containers boot); keep it running for operator live testing.
- **P9-T2** Session-resumption file (§11.4.131) + `docs/CONTINUATION.md` (§13.1) kept current; four-format exports where the doc class requires (§11.4.65/§11.4.153).

Design spikes to run before their phase’s line-level code (honesty — §11.4.6)

Spike	Why line-level detail waits	Phase
HelixLLM API contract (<code>API_CONTRACT.md</code>)	no canonical spec in-tree (DZ-05); gateway + banks depend on it	P1-T1 (first)
	new component, named by 4 streams, designed by none (DZ-04)	P1-T4

Spike	Why line-level detail waits	Phase
VRAM residency broker (VRAM_BROKER.md)		
A2A Go integration	verify latest A2A spec/SDK (§11.4.99) before coding	P4-T4

Governance-cascade follow-up (tracked, not blocking Phase 0)

Propagate §11.4.180–182 into helix_code CLAUDE.md/AGENTS.md/QWEN.md/GEMINI.md (§11.4.157) + bump the constitution submodule pointer in the same commit (CONST-049 step 7).